Lecture Note: Data Structures – Tuples & Dictionaries

Nishut Suman 8 Jan, 2026 At 11:00 AM Notes Foundation Recommended Resource

Associated Lectures (1)

Description Discussions

Lecture Note: Tuples and Dictionaries in Python

Prerequisites (Things to Know Before Reading)

Tuples in Python

Tuples are an essential data structure in Python, representing an **immutable**, **ordered collection** of elements. They are similar to lists but with a key difference: once a tuple is created, its contents cannot be changed. This immutability makes tuples a reliable choice for storing fixed collections of data.

Key Characteristics of Tuples

- Ordered (maintain insertion order)
- Immutable (cannot be changed after creation)
- Can contain elements of different data types
- Can be nested (tuples within tuples)
- Support indexing and slicing

Creating a Tuple

A tuple is created by placing all the items inside parentheses (), separated by commas. Tuples can hold elements of various data types.

```
# Empty tuple
tup = ()
print(tup)

# Using strings
tup = ('Python', 'Tuple')
print(tup)

# Using a list
```

```
li = [1, 2, 4, 5, 6]
print(tuple(li))

# Using the tuple() constructor
tup = tuple('Python')
print(tup)
```

Output:

```
()
('Python', 'Tuple')
(1, 2, 4, 5, 6)
('P', 'y', 't', 'h', 'o', 'n')
```

Creating a Tuple with Mixed Data Types

Tuples can contain elements of various data types, including other tuples, lists, and even functions.

```
tup = (5, 'Welcome', 7.5, True)
print(tup)

# Creating a Tuple with nested tuples
tup1 = (0, 1, 2, 3)
tup2 = ('Python', 'Data')
tup3 = (tup1, tup2)
print(tup3)

# Creating a Tuple with repetition
tup4 = ('Code',) * 3
print(tup4)

# Creating a Tuple using a loop
tup = ('Python')
n = 3
for i in range(n):
    tup = (tup,)
    print(tup)
```

Try running the above code and observe the output.

Are Tuples Immutable in Python?

Yes, tuples are immutable in Python. This means that once a tuple is created, its elements **cannot be changed, added, or removed**. This property distinguishes tuples from lists, which are mutable and allow modifications.

Example: Attempting to Modify a Tuple

```
# Creating a tuple
tup = (1, 2, 3, 4)

# Trying to change the third element
tup[2] = 99
```

Output:

```
TypeError: 'tuple' object does not support item assignment
```

Attempting to modify an element in a tuple results in an error, confirming that tuples are immutable.

Accessing Tuples

We can access tuple elements using **indexing** and **slicing**, similar to lists. Indexing starts at 0, and negative indices start from -1.

```
# Accessing Tuple with Indexing
tup = tuple("Python")
print(tup[0])

# Accessing a range of elements using slicing
print(tup[1:4])
print(tup[:3])

# Tuple unpacking
tup = ("Python", "is", "fun")
a, b, c = tup
print(a)
print(b)
print(c)
```

Output

```
P
('y', 't', 'h')
('P', 'y', 't')
Python
is
fun
```

Concatenation of Tuples

Tuples can be concatenated using the "+" operator, which creates a new tuple.

```
tup1 = (0, 1, 2, 3)
tup2 = ('Python', 'Tuple')

tup3 = tup1 + tup2
print(tup3)
```

Output:

```
(0, 1, 2, 3, 'Python', 'Tuple')
```

Slicing a Tuple

Slicing a tuple means creating a new tuple from a subset of the original elements.

```
tup = tuple('PYTHONCODE')

# Removing the first element
print(tup[1:])

# Reversing the Tuple
print(tup[::-1])

# Printing elements of a Range
print(tup[3:7])
```

Output:

```
('Y', 'T', 'H', 'O', 'N', 'C', 'O', 'D', 'E')
('E', 'D', 'O', 'C', 'N', 'O', 'H', 'T', 'Y', 'P')
('H', 'O', 'N', 'C')
```

Tuple Built-In Methods

Method	Description
`index()`	Returns the index of the specified element.

Method	Description
`count()`	Returns the number of times a value appears in the tuple.

Tuple Built-In Functions

Function	Description
`all()`	Returns True if all elements are true (or if tuple is empty).
`any()`	Returns True if any element is true.
`len()`	Returns the length of the tuple.
`enumerate()`	Returns an enumerate object for iteration.
`max()`	Returns the largest element in the tuple.
`min()`	Returns the smallest element in the tuple.
`sum()`	Returns the sum of numeric elements.
`sorted()`	Returns a sorted list of tuple elements.
`tuple()`	Converts an iterable to a tuple.

Python Dictionary

A **dictionary** in Python is a data structure that stores data in **key-value pairs**. Values can be of any data type, while keys must be **unique** and **immutable**.

Key Characteristics:

- Keys are case-sensitive.
- Duplicate keys overwrite previous values.
- Operations like search, insert, and delete are efficient (constant time).
- Ordered since Python 3.7.

Creating a Dictionary

Dictionaries can be created using curly braces {} or the dict() constructor.

```
# Using {}
d1 = {1: 'Python', 2: 'is', 3: 'Powerful'}
print(d1)
```

```
# Using dict() constructor
d2 = dict(a="Code", b="Data", c="AI")
print(d2)

# Accessing values
d = {"name": "John", 1: "Python", (1, 2): [1, 2, 4]}
print(d["name"])
print(d.get("name"))
```

Output:

```
{1: 'Python', 2: 'is', 3: 'Powerful'}
{'a': 'Code', 'b': 'Data', 'c': 'AI'}
John
John
```

Adding and Updating Dictionary Items

You can add new pairs or update existing ones using assignment.

```
d = {1: 'Python', 2: 'is', 3: 'Powerful'}

# Add new key-value pair
d["level"] = "Intermediate"

# Update existing value
d[1] = "Programming"

print(d)
```

Output:

```
{1: 'Programming', 2: 'is', 3: 'Powerful', 'level': 'Intermediate'}
```

Removing Dictionary Items

Dictionaries provide several methods for removing items.

```
d = {1: 'Python', 2: 'is', 3: 'Powerful', 'level': 'Intermediate'}

# Remove using del
del d["level"]
```

```
print(d)

# Remove using pop()
val = d.pop(1)
print(val)

# Remove using popitem()
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")

# Clear all items
d.clear()
print(d)
```

Output:

```
{1: 'Python', 2: 'is', 3: 'Powerful'}
Python
Key: 3, Value: Powerful
{}
```

Iterating Through a Dictionary

You can iterate through **keys**, **values**, or **key-value pairs**.

```
d = {1: 'Python', 2: 'is', 'level': 'Intermediate'}

# Iterate over keys
for key in d:
    print(key)

# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

Output:

```
1
2
level
Python
is
Intermediate
1: Python
```

```
2: is
level: Intermediate
```

Difference between List, Tuple, and Dictionary

List	Tuple	Dictionary
Stores elements in an ordered, mutable collection.	Stores elements in an ordered, immutable collection.	Stores data in key-value pairs.
Represented by `[ ]`	Represented by `( )`	Represented by `{ }`
Allows duplicate elements.	Allows duplicate elements.	Does not allow duplicate keys.
Can be nested.	Can be nested.	Can be nested.
Example: `[1, 2, 3]`	**Example:** `(1, 2, 3)`	**Example:** `{1: 'a', 2: 'b'}`
Created using `list()`	Created using `tuple()`	Created using `dict()`
Mutable	**Immutable**	**Mutable**
Ordered	Ordered	Ordered (Python 3.7+)
Empty list: `[]`	**Empty tuple:** `()`	**Empty dictionary:** `{}`

✅ Summary: What You Learned

By the end of this topic, you have learned:

- The definition, creation, and characteristics of **Tuples** in Python.
- Why tuples are **immutable** and how they differ from lists.
- Common **tuple methods** and built-in **functions**.
- How to **create**, **update**, **access**, and **iterate** through **Dictionaries**.
- Key **differences between Lists, Tuples, and Dictionaries**.

🎯 Final Thought

You've just explored two of Python's most powerful and widely used data structures - **Tuples** and **Dictionaries**.

Understanding their differences and strengths will help you write cleaner, faster, and more efficient Python code. Keep practicing by building small programs that use both together - that's where the real learning

happens!

- Practice Tip: Try running all the examples and code snippets in your notebook to get real-time experience and understand how the output changes as you modify the data. This hands-on practice will solidify your understanding of how tuples and dictionaries work.
-